Laporan Tugas Besar 1 IF3170 Inteligensi Artifisial
Optimasi Pemuatan Truk Peti Kemas 3D (Mejiro Express) Menggunakan Algoritma Local Search



Kelompok G46 - Clanker

13524002 - Daniel Anindito Nugroho
13524004 - Muhammad Fatih Irkham Mauludi
13524092 - Timothy Bernard Soeharto

Program Studi Teknik Informatika
STEI ITB
2026

Bab 1: Deskripsi Persoalan

1.1 Latar Belakang Persoalan

Mejiro Express memerlukan susunan paket berbentuk balok di dalam satu peti kemas truk yang memaksimalkan nilai barang terangkut. Setiap paket memiliki ukuran, berat, nilai, sifat pecah belah, dan ETA; tidak semua paket wajib dimuat. Pencarian lokal mengubah konfigurasi penempatan secara bertahap dengan tetap mempertimbangkan kendala geometri dan kapasitas [1], [2].

1.2 Daftar Variabel Permasalahan

Entitas	Atribut	Makna / representasi
Package	id	Identitas unik paket; kunci pemetaan paket.
Package	w, l, h	Dimensi awal balok; bilangan bulat.
Package / Placement	orientation	Indeks 0–5 pada Placement; menentukan permutasi dimensi efektif paket.
Package	value	Nilai paket yang dimaksimalkan.
Package	weight	Berat paket; dijumlahkan terhadap kapasitas truk.
Package	is_fragile	Penanda barang pecah belah yang tidak boleh menopang paket.
Package	eta	Sisa hari kedatangan; dipakai pada objektif alternatif.
Truck	w, l, h	Dimensi peti kemas.

Truck	max_capacity	Batas berat total paket di dalam truk.

1.3 Ketentuan Representasi State & Orientasi

State menyimpan Truck, kamus Package berdasarkan id, dan kamus Placement. Placement memuat package_id, position = (x, y, z), orientation, serta in_truck; koordinat menyatakan sudut minimum balok. Paket di luar diwakili oleh in_truck = False dan tidak ikut pemeriksaan geometri muatan; ekspor state mencatat identitasnya, bukan posisi fisik di luar kontainer.

Enam orientasi dimensi efektif ialah (w,l,h), (w,h,l), (l,w,h), (l,h,w), (h,w,l), dan (h,l,w). Model mengabaikan arah muka balok sehingga enam permutasi dimensi cukup untuk orientasi ortogonal. Implementasi rotate_package memilih indeks orientasi; perpindahan indeks secara umum belum memodelkan satu putaran 90° pada sumbu tertentu secara eksplisit.

Inisialisasi valid_floor mengacak urutan paket dan orientasi, lalu mencoba koordinat lantai berjarak sesuai dimensi paket dan menerima penempatan yang valid. Paket yang tidak dapat ditempatkan tetap di luar. Strategi dengan parameter *randomize* = True mengacak orientasi, koordinat, dan status masuk dengan tetap menjamin stabilitas, sedangkan selain itu akan memerikan *state* dengan nilai *value* paling rendah (dimana tidak ada barang) yang masuk ke truk.

1.4 Fungsi Objektif & Kendala (Constraints)

Untuk state valid, fungsi utama $F(s) = \Sigma\ value_i$ untuk seluruh paket dengan in_truck = True. State tidak valid mendapat skor −1000 × banyak pesan pelanggaran; jika mode strict = True, skor state tidak valid menjadi 0. Hill-Climbing membangkitkan kandidat valid sehingga eksperimen menggunakan kendala sebagai penyaring kelayakan.

Bonus 1 - Objektif alternatif. Digunakan beberapa heuristik lain berupa:

- Rasio volume antar truk, dimana fungsi objektif memaksimalkan rasio volume yang terpakai dibanding yang tidak terpakai
- Urgensi ETA, dimana fungsi objektif memaksimalkan banyaknya barang dengan ETA yang sekecil mungkin
- Kombinasi *value* dan rasio volume
- Kombinasi *value* dan urgensi ETA
- Kombinasi ketiga heuristik sebagai fungsi objektif

Perhitungan setiap fungsi objektif tersebut akan mempertimbangkan beberapa hal sebagai berikut:

Kendala	Aturan pemeriksaan
Batas kontainer	x,y,z ≥ 0; x+dx ≤ W, y+dy ≤ L, z+dz ≤ H. Seluruh paket aktif harus berada di dalam.
Bebas tabrakan	Irisan harus tidak memiliki volume positif pada ketiga sumbu sekaligus; bersentuhan muka diperbolehkan.
Tidak melayang	z = 0 atau ada paket lain dengan permukaan atas tepat di z serta irisan alas XY berluas positif. Tidak disyaratkan seluruh alas tertopang.
Pecah belah	Paket fragile tidak boleh memiliki paket lain tepat di atasnya dengan irisan XY.
Kapasitas berat	Jumlah berat paket aktif tidak melebihi max_capacity.

1.5 Operator Perpindahan Tetangga (Move Operators)

swap_packages menukar posisi, orientasi, dan status masuk dua paket; move_package_coordinate mengganti koordinat serta mengaktifkan status masuk; rotate_package mengganti indeks orientasi. toggle_in_truck menambah atau mengeluarkan paket dan membantu pencarian memilih subset muatan. Semua operator bekerja pada salinan state sehingga state asal tetap tersedia.

get_random_successor memilih operator acak dan, pada valid_only = True, mencoba hingga 50 kali untuk memperoleh kandidat valid. Koordinat disampel di lantai atau pada permukaan paket non-fragile dengan peluang memilih permukaan penopang 0,4 bila tersedia. generate_neighbors membuang kandidat yang sama dengan state asal, tetapi belum menghapus duplikasi antar kandidat.

Bab 2: Pembahasan Implementasi & Analisis Eksperimen

2.1 Implementasi Modul Core (src/core/)

2.1.1 models.py - model dan representasi

Package dan Truck merupakan dataclass frozen untuk data tetap; from_dict membaca atribut serta dimensi dari JSON. Package.get_rotated_dimensions memetakan indeks orientasi modulo enam ke dimensi efektif. Placement menyimpan data penempatan yang dapat berubah dan menyediakan copy untuk penyalinan.

```
def get_rotated_dimensions(self, orientation: int) -> Tuple[int, int, int]:
    w, l, h = self.w, self.l, self.h
    return ((w, l, h), (w, h, l), (l, w, h), (l, h, w), (h, w, l), (h, l,
w))[orientation % 6]
```

State.__post_init__ melengkapi placement yang belum ada sebagai paket di luar; get_package_dims dan get_bounding_box menghasilkan geometri efektif. Properti packages_in_truck, packages_outside_truck, total_weight_in_truck, dan total_value_in_truck menyajikan data turunan. clone menyalin placement sambil berbagi data paket/truk yang tetap, sedangkan to_dict mengekspor muatan, paket luar, total nilai, dan total berat.

```
def clone(self) -> State:
    return State(
        truck=self.truck,
        packages=self.packages,
        placements={k: v.copy() for k, v in self.placements.items()},
    )
```

2.1.2 validator.py - validasi geometri dan kapasitas

_placed_boxes mengambil bounding box paket aktif; _xy_overlap menguji irisan XY dengan ketaksamaan ketat. check_boundary_violations, check_overlap_violations, check_floating_violations, check_fragile_violations, dan check_capacity_violations masing-masing mengembalikan daftar pelanggaran. is_valid_state menggabungkan semuanya menjadi pasangan (status_valid, daftar_pelanggaran); pemeriksaan pasangan paket memiliki biaya $O(n^2)$.

```
supported = any(
    oid != pid and obox[2] + obox[5] == z and _xy_overlap(box, obox)
    for oid, obox in boxes
)
if not supported:
    violations.append(f"Package '{pid}' is floating at z={z}")
```

Kutipan di atas berasal dari check_floating_violations setelah kasus z = 0 dilewati. Dukungan parsial sesuai definisi tugas sudah cukup; validasi ini tidak mengukur kuat tekan, persentase bidang tumpu, maupun dinamika kendaraan.

2.1.3 objective.py - penilaian solusi

calculate_raw_value mengembalikan total nilai tanpa pemeriksaan kelayakan. calculate_objective memeriksa kendala sebelum menghitung nilai utama; calculate_alternative_objective menambahkan bonus volume, ETA, dan proksi stabilitas. Perbandingan eksperimen memakai fungsi utama, sehingga hasil 1.895 tidak menyatakan skor objektif alternatif.

```
is_valid, violations = is_valid_state(state)
if not is_valid:
    return -PENALTY_PER_VIOLATION * len(violations) if enforce_hard_constraints
else 0.0
return state.total_value_in_truck
eta_bonus += pkg.value * 0.15 / max(1, pkg.eta)
stability += (state.truck.h - p.position[2]) * (pkg.weight / max_cap) * 10.0
return state.total_value_in_truck + (used_vol / truck_vol) * 50.0 + eta_bonus +
stability
```

2.1.4 moves.py - operator dan pembangkitan kandidat

Empat operator perubahan dijelaskan pada Subbab 1.5. _sample_supported_coord memilih koordinat yang berpeluang mendapat penopang; validasi akhir tetap menentukan kelayakan karena sampling belum menjamin bebas tabrakan. generate_initial_state menyediakan dua strategi awal, sedangkan generate_neighbors menyusun sampel kandidat untuk algoritma pencarian.

```
def move_package_coordinate(state: State, pkg_id: str, new_coord: Tuple[int,
int, int]) -> State:
    new_state = state.clone()
    p = new_state.placements[pkg_id]
    p.position = new_coord
    p.in_truck = True
    return new_state
def rotate_package(state: State, pkg_id: str, new_orientation: Optional[int] =
None) -> State:
    new_state = state.clone()
    p = new_state.placements[pkg_id]
    p.orientation = new_orientation % 6 if new_orientation is not None else
(p.orientation + 1) % 6
    return new_state
```

2.2 Implementasi Algoritma Hill-Climbing (Bonus 2 - 4 Varian)

2.2.1 Struktur hasil dan runner

HillClimbingResult pada base.py menyimpan nama algoritma, state awal/akhir, objektif awal/akhir, durasi, total iterasi, riwayat objektif, dan extra_metrics. runner.main membaca argumen, memuat JSON, menjalankan varian berulang, memvalidasi state akhir, lalu menyimpan log. Seluruh varian menerima objective_fn sehingga fungsi penilaian dapat diganti tanpa mengubah alur pencarian.

```
res = steepest_ascent_hill_climbing(init_state, max_iterations=args.iterations)
is_valid, violations = is_valid_state(res.final_state)
```

2.2.2 Steepest Ascent Hill-Climbing - steepest_ascent.py

Varian ini menggunakan sideways_hill_climbing dengan max_sideways_moves = 0, sehingga hanya menerima kenaikan nilai. Pada setiap iterasi, algoritma memilih nilai tertinggi dari sampel tetangga valid dan memutus seri secara acak. Istilah steepest pada implementasi berarti terbaik di antara sampel, bukan enumerasi semua tetangga [3].

```
res = sideways_hill_climbing(
    initial_state=initial_state,
    max_sideways_moves=0,
    max_iterations=max_iterations,
    sample_neighbors=sample_neighbors,
    objective_fn=objective_fn,
)
res.algorithm_name = "Steepest Ascent Hill-Climbing"
res.extra_metrics = {"iterations_to_stop": res.total_iterations}
```

2.2.3 Stochastic Hill-Climbing - stochastic.py

Stochastic menyaring kandidat yang memiliki nilai lebih tinggi daripada state saat ini, lalu memilih satu secara acak dengan bobot kenaikan nilainya. Kandidat dengan perbaikan besar memiliki peluang lebih tinggi, tetapi kandidat lain yang membaik tetap dapat terpilih. Pencarian berhenti ketika sampel tidak memiliki perbaikan atau batas iterasi tercapai.

```
improving = [(n, val, val - current_val) for n in neighbors if (val :=
objective_fn(n)) > current_val]
if not improving:
    break
weights = [delta for _, _, delta in improving]
current_state, current_val, _ = random.choices(improving, weights=weights,
k=1)[0]
history.append(current_val)
```

2.2.4 Hill-Climbing with Sideways Move - sideways_move.py

Algoritma memilih tetangga dengan skor maksimum. Kenaikan skor diterima dan mereset penghitung sideways berturut-turut; skor sama dapat diterima selama penghitung tersebut belum mencapai max_sideways_moves. Perubahan susunan tanpa mengubah nilai dapat membuka penempatan paket tambahan pada langkah berikutnya, tetapi belum menjamin keluar dari setiap plateau.

```
if best_val > current_val:
    current_state = random.choice(best_neighbors)
    current_val = best_val
    consecutive_sideways = 0
    history.append(current_val)
elif best_val == current_val and consecutive_sideways < max_sideways_moves:
    consecutive_sideways += 1
    total_sideways += 1
    current_state = random.choice(best_neighbors)
    history.append(current_val)
else:
    break
```

Parameter bawaan max_sideways_moves = 50 membatasi langkah setara secara berturut-turut, bukan total sepanjang run. Karena penghitung direset setelah perbaikan, total_sideways_moves_taken dapat melebihi 50. Iterasi penghentian tetap dihitung meskipun tidak menghasilkan perpindahan state.

2.2.5 Random Restart Hill-Climbing - random_restart.py

Random Restart menjalankan Steepest Ascent dari beberapa state awal valid_floor yang berbeda dan mempertahankan solusi terbaik di seluruh percobaan. Pada kode ini max_restarts = 5 berarti lima pencarian termasuk pencarian pertama, sehingga terdapat empat pengawalan ulang setelahnya. total_iterations merupakan jumlah iterasi semua pencarian dan iterations_per_restart mencatat rinciannya.

```
for r in range(max_restarts):
       init_state = first_state if r == 0 else generate_initial_state(truck,
packages, random_strategy="valid_floor")
    run = steepest_ascent_hill_climbing(
        initial_state=init_state,
        max_iterations=max_iterations_per_restart,
        sample_neighbors=sample_neighbors,
        objective_fn=objective_fn,
    )
    iters_per_restart.append(run.total_iterations)
    combined_history.extend(run.objective_history[1:])
    if run.final_objective > best_val:
        best_val = run.final_objective
        best_state = run.final_state
```

combined_history menggabungkan langkah yang diterima dari tiap pencarian tanpa menyisipkan seluruh skor awal restart. Akibatnya, elemen terakhir riwayat tidak selalu sama dengan best_val. Hasil akhir dalam tabel diambil dari final_state.total_value; plot nantinya perlu membedakan riwayat setiap restart dan kurva solusi terbaik.

2.3 Implementasi Algoritma Simulated Annealing

2.3.1 Representasi, pembangkitan tetangga, dan penerimaan solusi

Implementasi Simulated Annealing (SA) beroperasi pada struktur data SolverState yang merepresentasikan konfigurasi penempatan seluruh paket di dalam dan di luar ruang peti kemas. Pembangkitan solusi tetangga menggunakan fungsi get_random_successor(state, valid_only=True) dari modul src/core/moves.py. Fungsi ini memilih secara acak salah satu operator perpindahan: penukaran dua paket (swap), pemindahan koordinat di dalam truk (move_in_truck), pemindahan paket antar-truk atau keluar-masuk truk (move_between_truck), dan rotasi orientasi 90 derajat (rotate). Setiap kandidat tetangga divalidasi langsung oleh mesin kendala 3D (validate_state) untuk memastikan tidak melanggar batas ruang peti kemas, tidak tumpang tindih, memiliki dukungan gravitasi pada bidang alas, mematuhi batasan penumpukan barang pecah belah (fragile), serta tidak melebihi kapasitas beban maksimum truk.

Evaluasi kualitas solusi menggunakan fungsi calculate_objective, di mana nilai objektif merepresentasikan total nilai paket yang termuat di dalam truk dikurangi penalti pelanggaran. Pada setiap iterasi, dihitung selisih nilai objektif antara kandidat tetangga dan solusi saat ini: delta_E = f(S_neighbor) - f(S_current). Kriteria penerimaan solusi mengadopsi mekanisme probabilitas Metropolis. Jika delta_E > 0, solusi tetangga memberikan peningkatan nilai dan langsung diterima. Jika delta_E <= 0, solusi yang lebih buruk tetap dapat diterima dengan probabilitas P = exp(delta_E / max(T_min, T)). Algoritma membangkitkan bilangan acak seragam r pada rentang [0, 1); jika r < P, mutasi tersebut diterima. Mekanisme penerimaan stokastik ini memungkinkan pencarian melompat keluar dari cekungan optimum lokal pada tahap awal iterasi.

2.3.2 Parameter temperatur, cooling schedule, dan source code

Penurunan temperatur diatur menggunakan skema pendinginan geometrik: T_(k+1) = alpha * T_k. Parameter bawaan yang digunakan adalah temperatur awal T_0 = 1000,0 (DEFAULT_INITIAL_TEMP), laju pendinginan alpha = 0,95 (DEFAULT_COOLING_RATE), dan ambang temperatur batas bawah T_min = 1e-4 (MINIMAL_TEMPERATURE). Pencarian berhenti apabila temperatur mencapai T <= T_min atau telah mencapai batas 200 iterasi.

```
   def simulated_annealing(
    initial_state: SolverState,
    initial_temp: float = DEFAULT_INITIAL_TEMP,
    cooling_rate: float = DEFAULT_COOLING_RATE,
    max_iterations: int = -1,
    strict: bool = False,
    objective_type: str = "default",
) -> Generator[Dict[str, Any], None, None]:
    current_state = initial_state.clone()
```

```
    current_val = calculate_objective(current_state, strict=strict,
objective_type=objective_type)
    temp = float(initial_temp)
    iteration = 0

    while (max_iterations == -1 or iteration < max_iterations) and temp >
MINIMAL_TEMPERATURE:
        iteration += 1
        neighbor = get_random_successor(current_state, valid_only=True)
        neighbor_val = calculate_objective(neighbor, strict=strict,
objective_type=objective_type)
        delta_e = neighbor_val - current_val

        if delta_e > 0:
            prob = 1.0
            accepted = True
        else:
            prob = math.exp(delta_e / max(MINIMAL_TEMPERATURE, temp))
            accepted = random.random() < prob

        if accepted:
            current_state = neighbor
            current_val = neighbor_val

        temp *= cooling_rate
        yield {
            "iteration": iteration,
            "objective_value": current_val,
            "state": {k: v.to_dict() for k, v in current_state.placements.items()},
            "temperature": temp,
            "delta_e": delta_e,
            "acceptance_chance": prob,
            "is_stuck": int(not accepted),
        }
```

2.4 Implementasi Genetic Algorithm

2.4.1 Representasi kromosom dan inisialisasi populasi

Individual membungkus State sebagai kromosom: setiap gen menyimpan Placement satu paket, sedangkan fitness menyimpan nilai objektif. evaluate() mencatat validitas dan pelanggaran sebelum menghitung fitness; create_random() menggunakan generate_initial_state dengan strategi valid_floor, lalu mengevaluasi individu. clone() menyalin state agar perubahan keturunan tidak mengubah induk.

```
    def evaluate(self, objective_func: Callable[[State], float] =
calculate_objective) -> float:
        """Menghitung nilai fitness individu berdasarkan fungsi objektif"""
        self.is_valid, self.violations = is_valid_state(self.state)
        self.fitness = objective_func(self.state)
        return self.fitness
```

2.4.2 Seleksi, crossover, mutasi, dan penggantian generasi

tournament_selection memilih individu terbaik dari tiga kandidat acak, sedangkan roulette_wheel_selection memakai bobot fitness − fitness minimum + 1 agar semua bobot positif. uniform_crossover menukar asal gen tiap paket dengan peluang 50%; two_point_crossover menukar segmen di antara dua titik. mutate_individual memilih swap, perpindahan koordinat/status muat, atau rotasi; keturunan tidak diperbaiki otomatis, sehingga pelanggaran dikenai penalti objektif. Elitism menyalin dua individu terbaik (elitism_count=2) ke generasi berikutnya.

```
r = rng if rng is not None else random
k = min(tournament_size, len(population))
candidates = r.sample(population, k)
return max(candidates, key=lambda ind: ind.fitness).clone()
```

```
next_gen: List[Individual] = [
    population[i].clone() for i in range(min(elitism_count, len(population)))
]
```

2.4.3 Parameter populasi, iterasi, dan source code

genetic_algorithm() membentuk populasi, mengevaluasi dan mengurutkannya, lalu mengulang seleksi, crossover, mutasi, dan penggantian generasi. Nilai default ialah population_size=50, max_generations=100, crossover_rate=0.8, mutation_rate=0.2, seleksi tournament, serta crossover uniform; hasil dikembalikan sebagai AlgorithmResult. best_history dan avg_history dicatat sebelum reproduksi dan disimpan sebagai objective_history serta avg_fitness_history: titik pertama mewakili populasi awal, sedangkan populasi setelah reproduksi terakhir belum masuk grafik.

```
    for gen in range(max_generations):
        population.sort(key=lambda x: x.fitness, reverse=True)
        current_best = population[0].fitness
        current_avg = sum(x.fitness for x in population) / len(population)
        best_history.append(current_best)
        avg_history.append(current_avg)
```

Benchmark CLI melalui run_single_algorithm() dan run_experiment() menyeragamkan pemanggilan, pengulangan, dan log state/objektif/durasi. ga_matrix_runner menjalankan dua kelompok eksperimen, masing-masing tiga konfigurasi × tiga pengulangan, lalu menghasilkan ringkasan JSON dan grafik. Seed operator tersedia, tetapi rng pada create_random() belum diteruskan ke generator state; karena itu seed saja belum menjamin populasi awal yang identik.

```
python -m src.experiments.ga_matrix_runner
```

2.5 Implementasi Visualisasi Program

2.5.1 Tampilan truk, paket di dalam/luar, serta penanda fragile

Visualisasi 3D interaktif diimplementasikan pada modul src/app/scene.py menggunakan pustaka Plotly (go.Mesh3d dan go.Scatter3d). Setiap kontainer truk dimodelkan dengan bidang alas jalan aspal, sasis, roda ban, serta rangka peti kemas transparan dengan garis kawat (wireframe). Di atas setiap truk terdapat label teks melayang yang menampilkan dimensi fisik, akumulasi berat muatan, batas kapasitas maksimum, persentase utilisasi, dan indikator warna status beban (hijau jika muatan aman dan merah jika terjadi kelebihan beban).

Paket yang dimuat di dalam truk ditampilkan sebagai balok pejal (opasitas 0,90) dengan garis tepi kawat yang tegas untuk memperjelas susunan antar-paket. Paket yang belum dimuat dialokasikan ke area tunggu di belakang truk, yaitu STAGING AREA (UNPACKED) pada koordinat sumbu x melebihi panjang truk ditambah 3,0 unit di atas palet platform oranye dengan tampilan semi-transparan (opasitas 0,65). Paket pecah belah (is_fragile = True) diberi penanda khusus berupa garis tepi putus-putus merah terang serta label peringatan fragile pada tooltip interaktif saat objek disorot kursor.

Implementasi dilakukan dengan detail file-file berikut:

- **app.py** (*entrypoint* aplikasi *webapp* yang dapat dijalankan)
- **interface.py** (logika *rendering frontend* yang menyusun setiap komponen)
- **scene.py** (logika *rendering* grafik 2D dan 3D yang terintegrasi melalui *library* Dash)
- **callback.py** (logika *script frontend* yang akan ditranslasikan menjadi JavaScript untuk komunikasi antara *client* dan *server*)
- **assets/custom.css** (*stylesheet* untuk bergagai ikon dan warna)

2.5.2 Tampilan initial/final state dan grafik eksperimen

plot_ga_convergence() memakai Matplotlib untuk menggambar fitness maksimum dan rata-rata populasi terhadap generasi, lalu menyimpan gambar tanpa GUI melalui backend Agg. plot_parameter_analysis() membuat dua panel: variasi generasi pada populasi tetap dan variasi populasi pada generasi tetap; tiap panel membandingkan rata-rata objektif akhir dan durasi dengan dua sumbu vertikal.

```
    ax.plot(
        iterations,
        history_max,
        label="Fitness Maksimum (Best)",
        color="#1f77b4",
```

```
        linewidth=2.2,
    )
    ax.plot(
        iterations,
        history_avg,
        label="Fitness Rata-rata Populasi (Avg)",
        color="#ff7f0e",
        linewidth=1.8,
        linestyle="--",
    )
```

2.6 Hasil dan Analisis Eksperimen Hill-Climbing

2.6.1 Konfigurasi eksperimen

Pengujian dilakukan pada data/sample_input.json: 12 paket, kontainer 10 × 20 × 5, dan kapasitas berat 500. Setiap varian dijalankan tiga kali pada Python lokal dengan fungsi utama, state awal valid_floor, batas 200 iterasi, batas 50 sideways berturut-turut, serta lima percobaan Random Restart. Steepest, Stochastic, dan Sideways menyampel 40 tetangga per iterasi; Random Restart menyampel 30.

```
python -m src.algorithms.hill_climbing.runner --runs 3
```

Perintah dijalankan, kemudian pengambilan data diulang dengan parameter pencarian identik dan nama log unik untuk menjaga seluruh rekaman run. Tabel berikut hanya memakai rangkaian pengambilan data kedua, bukan memilih hasil terbaik dari dua rangkaian. Seed tidak ditetapkan dan state awal antarrun/varian berbeda; durasi memakai perf_counter untuk pencarian saja, di luar pembacaan berkas dan penyimpanan log.

2.6.2 Hasil tiga run setiap varian

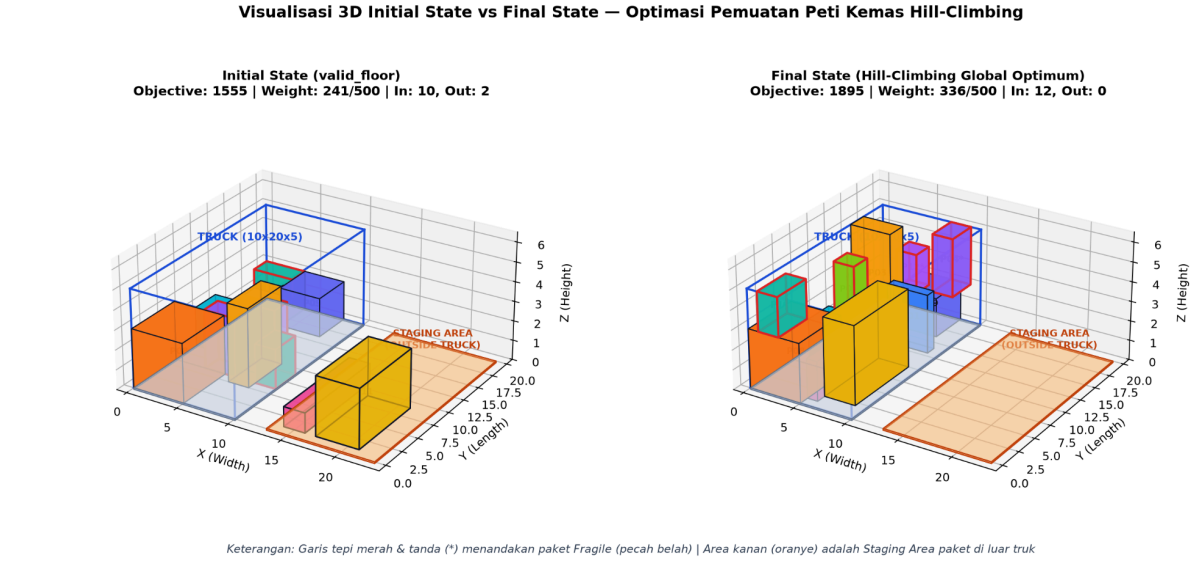
Varian	Run	F awal	F akhir	Durasi (s)	Iterasi	Metrik khusus
Steepest	1	1285	1635	0,011963	2	-
Steepest	2	1545	1545	0,006523	1	-
Steepest	3	1545	1545	0,005641	1	-

Stochastic	1	1285	1285	0,005169	1	-
Stochastic	2	1695	1775	0,012754	2	-
Stochastic	3	1285	1285	0,004681	1	-
Sideways	1	1555	1895	0,532396	64	59 sideways
Sideways	2	1815	1895	0,481089	53	51 sideways
Sideways	3	1465	1895	0,633115	63	59 sideways
Restart	1	1555	1895	0,024436	5	5 percobaan; [1,1,1,1,1]
Restart	2	1555	1635	0,027649	6	5 percobaan; [2,1,1,1,1]
Restart	3	1285	1815	0,040770	8	5 percobaan; [1,2,1,1,3]

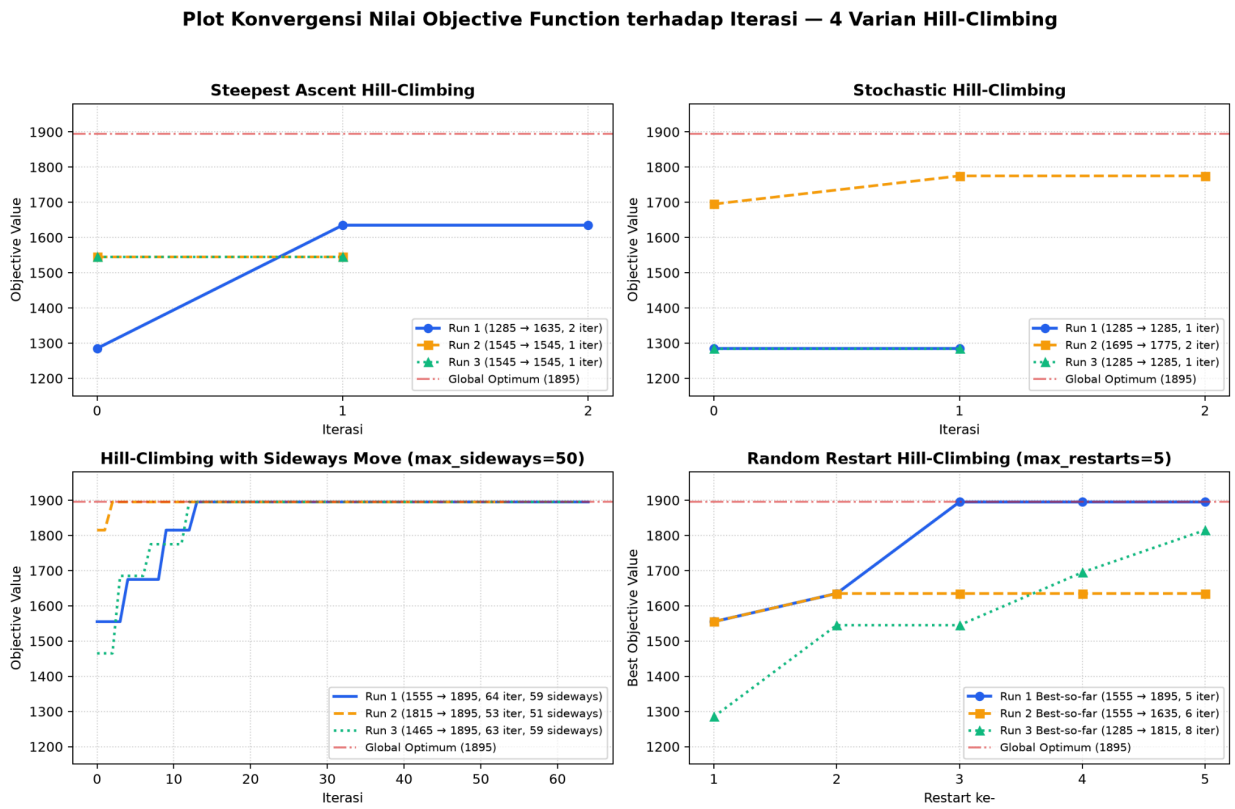
Seluruh 12 state akhir valid menurut kelima kendala. Kolom iterasi mencakup pemeriksaan terakhir yang memicu penghentian; untuk Restart, nilainya adalah jumlah iterasi lima pencarian. Daftar dalam kurung siku menunjukkan iterasi tiap percobaan, dan jumlah pengawalan ulang setelah percobaan pertama adalah empat.

Varian	Rata-rata F akhir	Rentang F akhir	Rata-rata waktu (s)	Rata-rata iterasi
Steepest	1575,00	1545–1635	0,008042	1,33
Stochastic	1448,33	1285–1775	0,007535	1,33

Sideways	1895,00	1895–1895	0,548867	60,00
Random Restart	1781,67	1635–1895	0,030952	6,33



Keterangan: Garis tepi merah & tanda (*) menandakan paket Fragile (pecah belah) | Area kanan (oranye) adalah Staging Area paket di luar truk



2.6.3 Kedekatan terhadap global optimum

Jumlah nilai seluruh paket pada sampel adalah 1.895, sehingga nilai tersebut merupakan batas atas fungsi utama. Sideways mencapainya pada ketiga run dan Random Restart pada run pertama dengan state valid; karena semua paket bernilai positif telah terangkut, hasil tersebut sekaligus membuktikan optimum global untuk kasus sampel dan fungsi utama. Pernyataan ini tidak berlaku otomatis untuk masukan lain atau objektif alternatif.

2.6.4 Plateau dan peran sideways move

Steepest dan Stochastic berhenti setelah satu atau dua iterasi, sedangkan Sideways melakukan 51–59 langkah setara dan mencapai 1.895 pada semua run. Nilai utama tidak berubah ketika paket hanya dipindah atau dirotasi, sehingga langkah setara dapat memperbaiki ruang kosong sebelum paket tambahan masuk. Berhenti tanpa perbaikan pada sampel tetangga belum membuktikan local maximum pada seluruh neighborhood; plateau dan tetangga yang tidak tersampel tetap mungkin.

2.6.5 Peran random restart

Lima state awal memperluas wilayah pencarian tanpa menerima langkah menurun dalam satu pencarian. Random Restart memperoleh rata-rata 1.781,67, lebih tinggi daripada Steepest (1.575,00) dan Stochastic (1.448,33) dalam rangkaian ini, tetapi tidak selalu mencapai optimum.

Run pertama memperlihatkan manfaat state awal baru: semua pencarian berhenti pada satu iterasi, namun solusi terbaik meningkat dari 1.555 menjadi 1.895.

2.6.6 Konsistensi dan durasi

Sideways paling konsisten pada tiga pengulangan ini (rentang nilai akhir nol), dengan waktu rata-rata sekitar 0,549 detik. Steepest dan Stochastic hanya memerlukan sekitar 0,008 detik, sedangkan Random Restart sekitar 0,031 detik. Selisih ini sejalan dengan banyaknya iterasi, tetapi tiga run dengan state awal berbeda dan ukuran sampel tetangga yang tidak sama belum cukup untuk menyimpulkan keunggulan umum atau perbandingan waktu yang terkendali.

2.6.7 Batas pencatatan dan tindak lanjut

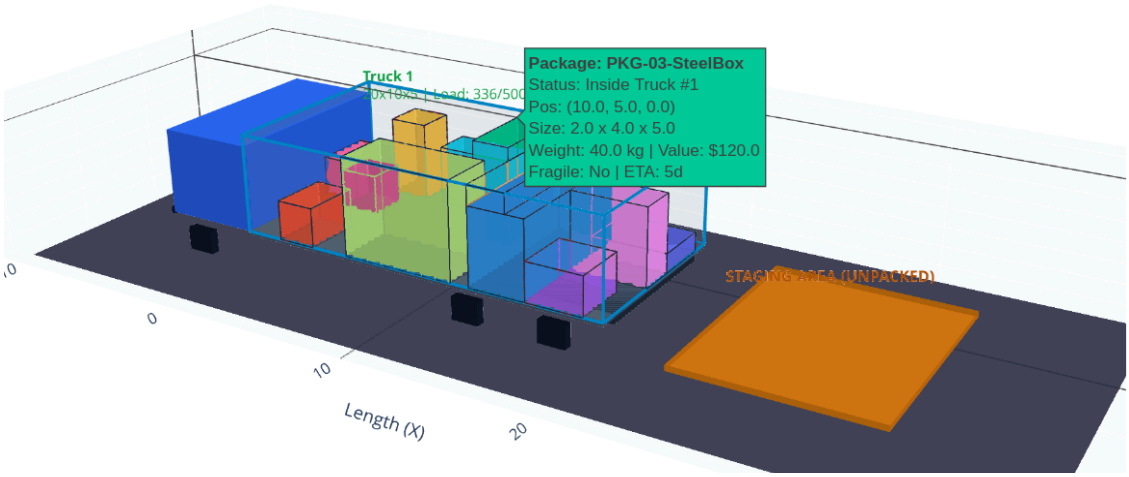
Log bawaan dapat tertimpa akibat resolusi nama berkas satu detik. Selain itu, pada Restart run pertama, elemen akhir objective_history bernilai 1.555 sementara final_state bernilai 1.895; karena save_run_output mengambil final_objective dari riwayat, kolom itu tidak dipakai untuk penilaian akhir laporan. Perbaikan penamaan log, pencatatan best-so-far, seed tetap, dan masukan yang lebih padat diperlukan untuk eksperimen lanjutan.

2.7 Hasil dan Analisis Eksperimen Simulated Annealing

2.7.1 Konfigurasi dan tabel tiga run

Pengujian Simulated Annealing dilakukan tiga kali pada data/sample_input.json (12 paket, kontainer 10 x 20 x 5, kapasitas berat 500 unit). Parameter pencarian meliputi T_0 = 1000, alpha = 0,95, T_min = 1e-4, dan batas 200 iterasi. State awal dibangkitkan menggunakan strategi valid_floor. Waktu komputasi diukur murni pada fase pencarian lokal menggunakan time.perf_counter().

Run	F Awal	F Akhir	Iterasi	Total Stuck	Rasio Stuck	Validitas	Durasi (s)
1	1895	1820	200	63	31,5%	Valid	0,0606
2	1895	1815	200	65	32,5%	Valid	0,0512
3	1795	1815	200	58	29%	Valid	0,0899
Rata-rata	1861,67	1816,67	200	62	31%	100%	0,0672



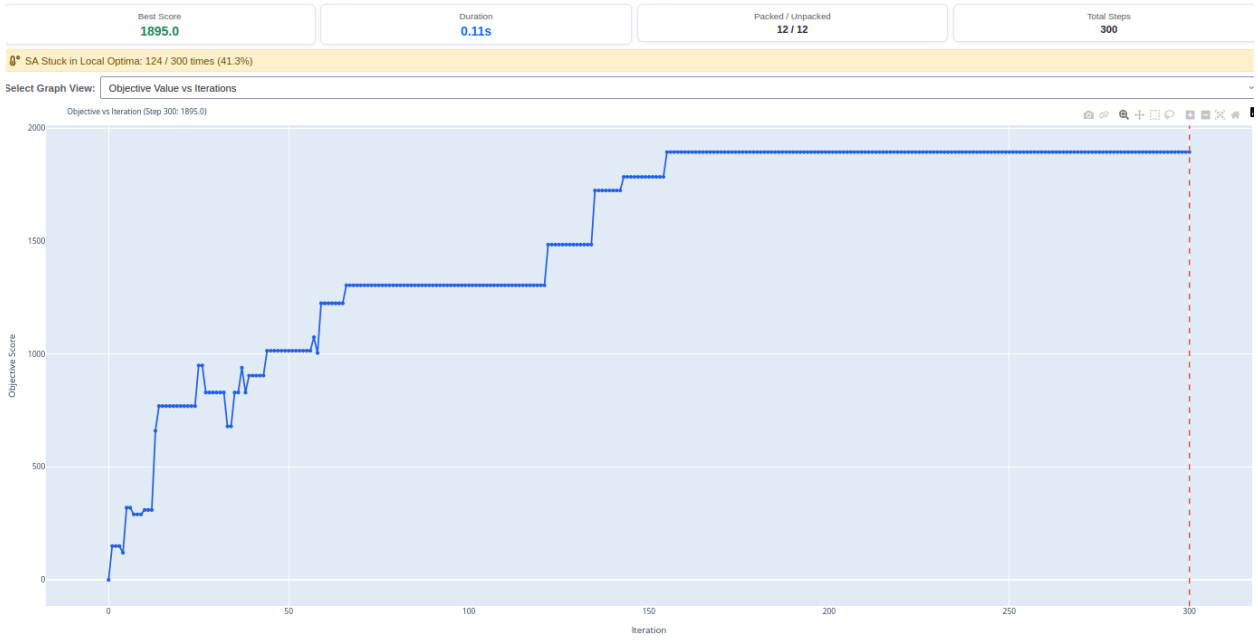
2.7.2 State awal/akhir, konvergensi, dan probabilitas penerimaan

Seluruh state akhir pada ketiga run berhasil memenuhi seluruh kendala fisik 3D dan batasan berat secara valid. Konvergensi nilai objektif berhubungan langsung dengan dinamika probabilitas penerimaan Metropolis P = exp(delta_E / T). Pada iterasi 1 sampai 50 dengan rentang temperatur 1000 hingga 76,9, nilai probabilitas penerimaan untuk langkah yang menurunkan objektif berkisar antara 0,70 hingga 0,98. Algoritma secara aktif melompat antar-state dan mengeksplorasi ruang pencarian. Pada iterasi 51 sampai 120 dengan temperatur 76,9 hingga 2,0, probabilitas penerimaan langkah buruk turun ke kisaran 0,10 hingga 0,45 sehingga degradasi besar mulai ditolak. Pada iterasi 121 sampai 200 dengan temperatur di bawah 1,0, probabilitas penerimaan langkah buruk mendekati nol, menyebabkan algoritma bertransisi menjadi eksploitasi murni mirip Hill-Climbing deterministik yang mempertahankan konfigurasi terbaik pada cekungan lokal tersebut.



2.7.3 Frekuensi stuck, kualitas solusi, konsistensi, dan durasi

Metrik stuck mencatat frekuensi penolakan langkah ketika kandidat tidak menghasilkan peningkatan dan gagal melampaui ambang batas acak probabilitas Metropolis. Rasio stuck konsisten pada rentang 29,0% hingga 32,5% dengan rata-rata 31,00%. Penolakan langkah terkonsentrasi pada sepertiga akhir iterasi saat temperatur sudah mendingin di bawah 1,0. Durasi rata-rata sebesar 0,0672 detik menunjukkan bahwa algoritma bekerja jauh lebih cepat dibandingkan HC Sideways Move (0,5489 detik) dan GA (0,5511 hingga 1,0200 detik). Nilai akhir rata-rata 1816,67 mendekati batas optimum global 1895,00 dengan selisih 4,13%. Penurunan nilai dari inisialisasi awal pada Run 1 dan 2 terjadi karena algoritma mengembalikan state posisi terakhir pada iterasi ke-200, bukan memori state terbaik yang sempat dilalui pada fase suhu tinggi.



2.8 Hasil dan Analisis Eksperimen Genetic Algorithm

2.8.1 Populasi tetap dan tiga variasi iterasi (9 run)

Masukan: data/sample_input.json (12 paket), objektif utama; crossover 0,8, mutasi 0,2, tournament ukuran 3, uniform crossover, dan dua elite. Nilai awal adalah individu terbaik populasi awal; seluruh 18 state akhir lolos validator. Durasi diukur dalam detik, termasuk pembentukan populasi GA; semua run menyelesaikan G generasi.

Run	N	G	Ulangan	Awal	Akhir	Durasi (s)
1	50	50	1	1895	1895	0,3683

2	50	50	2	1895	1895	0,3421
3	50	50	3	1815	1815	0,3319
4	50	100	1	1895	1895	0,8374
5	50	100	2	1895	1895	1,1237
6	50	100	3	1895	1895	1,0988
7	50	200	1	1820	1895	2,2948
8	50	200	2	1895	1895	2,1226
9	50	200	3	1895	1895	2,2265

Konfigurasi	Rerata akhir	SD akhir	Rerata s	SD s	Optimum
G=50	1868,33	46,19	0,3474	0,0188	2/3
G=100	1895,00	0,00	1,0200	0,1586	3/3
G=200	1895,00	0,00	2,2146	0,0867	3/3

SD adalah simpangan baku sampel (n−1) dari tiga pengulangan; statistik durasi dihitung dari log yang dibulatkan empat desimal. Kedua kelompok merupakan run terpisah, termasuk konfigurasi N=50 dan G=100 yang muncul pada keduanya.

2.8.2 Iterasi tetap dan tiga variasi populasi (9 run)

Masukan: data/sample_input.json (12 paket), objektif utama; crossover 0,8, mutasi 0,2, tournament ukuran 3, uniform crossover, dan dua elite. Nilai awal adalah individu terbaik populasi awal; seluruh 18 state akhir lolos validator. Durasi diukur dalam detik, termasuk pembentukan populasi GA; semua run menyelesaikan G generasi.

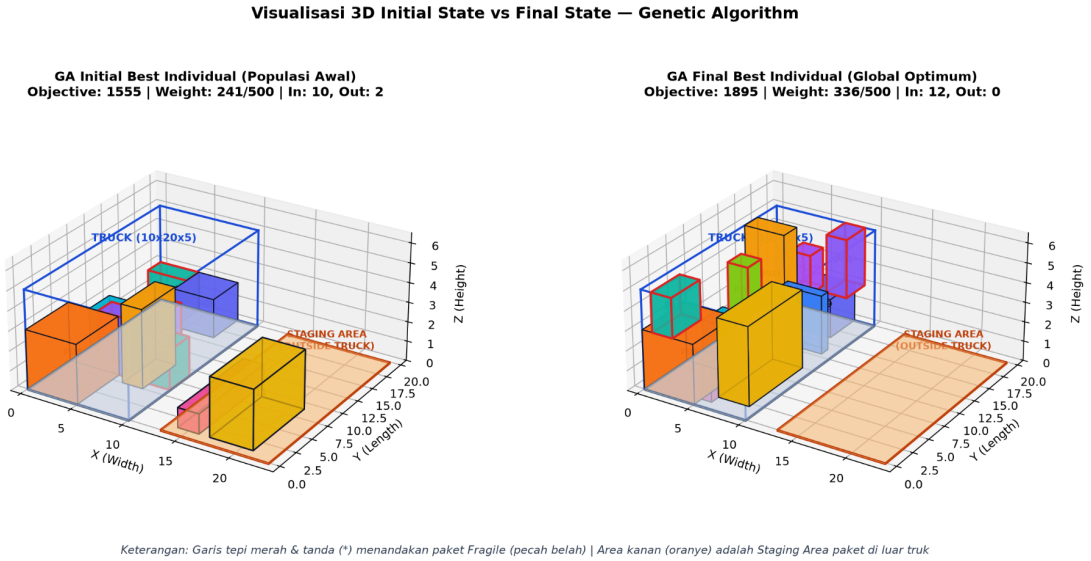
Run	N	G	Ulangan	Awal	Akhir	Durasi (s)
10	20	100	1	1895	1895	0,5178
11	20	100	2	1815	1895	0,4593

12	20	100	3	1895	1895	0,6762
13	50	100	1	1895	1895	1,7323
14	50	100	2	1895	1895	1,4890
15	50	100	3	1895	1895	1,1898
16	100	100	1	1895	1895	3,8374
17	100	100	2	1895	1895	2,5175
18	100	100	3	1895	1895	2,6974

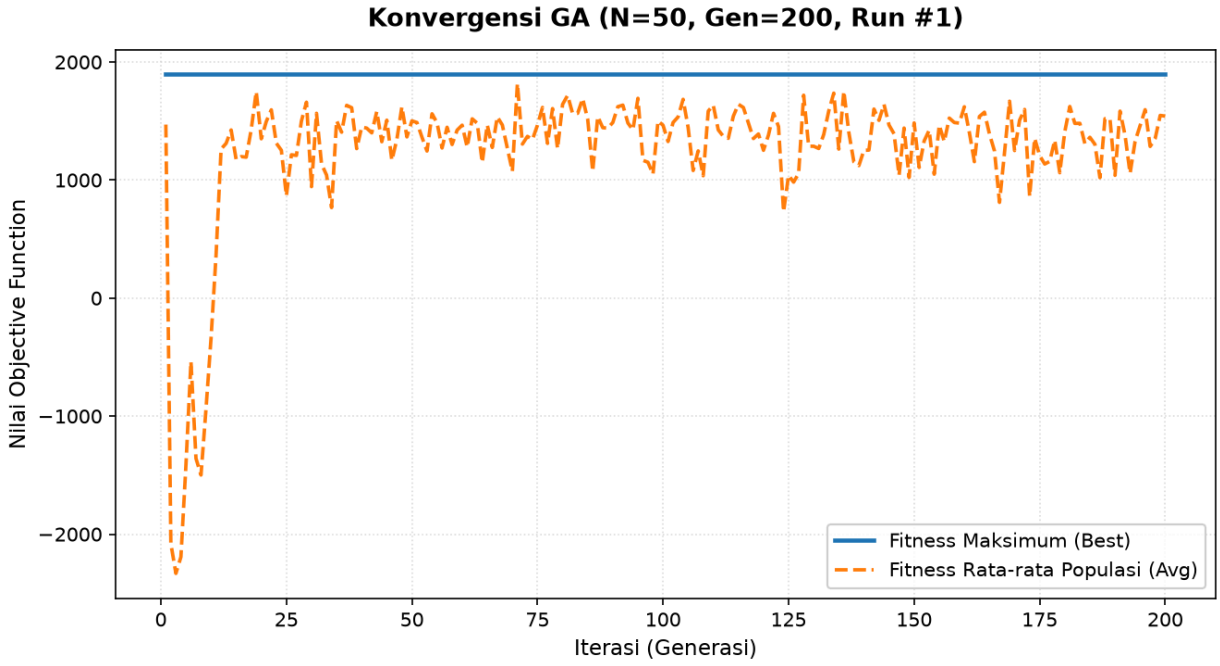
Konfigurasi	Rerata akhir	SD akhir	Rerata s	SD s	Optimum
N=20	1895,00	0,00	0,5511	0,1122	3/3
N=50	1895,00	0,00	1,4704	0,2717	3/3
N=100	1895,00	0,00	3,0174	0,7158	3/3

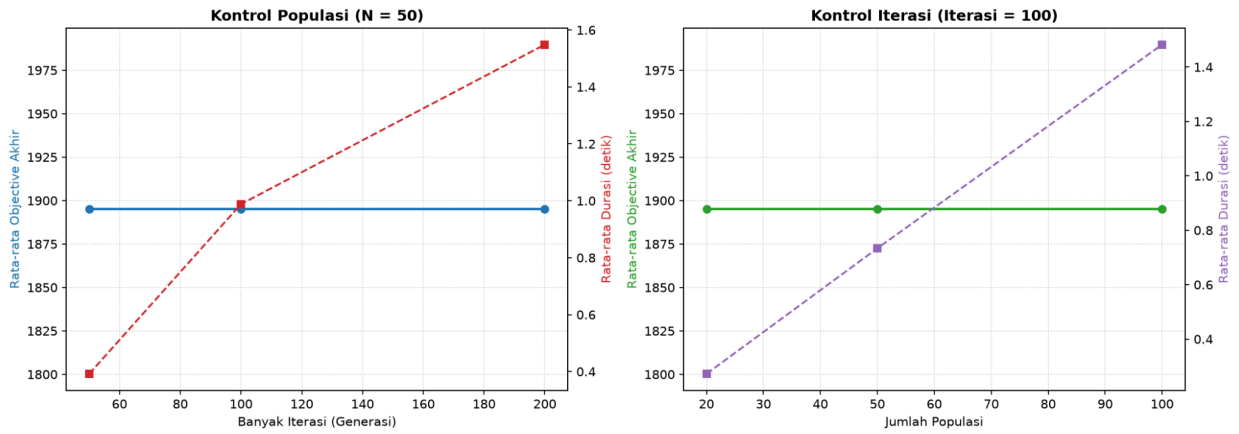
SD adalah simpangan baku sampel (n−1) dari tiga pengulangan; statistik durasi dihitung dari log yang dibulatkan empat desimal. Kedua kelompok merupakan run terpisah, termasuk konfigurasi N=50 dan G=100 yang muncul pada keduanya.

2.8.3 State awal/akhir, grafik maksimum/rata-rata, dan analisis parameter



Keterangan: Garis tepi merah & tanda (*) menandakan paket Fragile (pecah belah) | Area kanan (oranye) adalah Staging Area paket di luar truk





Batas atas nilai muatan adalah 1.895, yaitu jumlah nilai seluruh paket; solusi valid yang mencapainya membuktikan optimum global pada sampel ini. GA mencapainya pada 17/18 run, tetapi 15/18 populasi awal sudah memiliki individu optimum. Run 7 meningkat dari 1.820 menjadi 1.895 pada indeks riwayat 8, run 11 dari 1.815 menjadi 1.895 pada indeks 15, sedangkan run 3 bertahan pada 1.815.

Pada N=50, penambahan G dari 50 ke 100 meningkatkan keberhasilan dari 2/3 ke 3/3; G=200 tetap 3/3 dengan durasi rata-rata naik dari 0,3474 menjadi 1,0200 dan 2,2146 detik. Pada G=100, N=20, 50, dan 100 sama-sama mencapai optimum 3/3, dengan durasi 0,5511; 1,4704; dan 3,0174 detik. Ukuran populasi lebih besar belum memberi peningkatan kualitas pada sampel ini; tiga pengulangan dan populasi awal yang berbeda membatasi kesimpulan kausal.

Elitism menjaga fitness terbaik, tetapi rata-rata populasi berfluktuasi dan tidak konvergen ke 1.895. Sebagai contoh, pada N=50 dan G=100 (run 4–6), rerata fitness populasi turun dari 1.515,53 ke 1.116,63 karena keturunan hasil crossover/mutasi dapat melanggar kendala dan terkena penalti. Grafik maksimum yang mendatar sejak awal terutama mencerminkan optimum yang sudah ditemukan saat inisialisasi, bukan bukti percepatan pencarian oleh evolusi.

2.9 Analisis Perbandingan Lintas Algoritma (HC vs SA vs GA)

Pada sampel yang sama, Sideways dan GA (N=50, G=100; run 4–6) sama-sama mencapai optimum 1.895 pada 3/3 run. Steepest, Stochastic, dan Random Restart memiliki rata-rata akhir lebih rendah; Random Restart mencapai optimum sekali. GA N=20, G=100 juga optimum 3/3 dengan durasi rata-rata 0,5511 detik, hampir sama dengan Sideways 0,5489 detik.

Algoritma	Rerata akhir	Optimum	Rerata durasi (s)
HC Steepest	1.575,00	0/3	0,0080

HC Stochastic	1.448,33	0/3	0,0075
HC Sideways	1.895,00	3/3	0,5489
HC Random Restart	1.781,67	1/3	0,0310
Simulated Annealing	1816,67	0/3	0,0672
GA N=50, G=100	1.895,00	3/3	1,0200

GA konfigurasi pembanding memerlukan sekitar 1,86 kali durasi Sideways. Perbandingan ini bersifat deskriptif: eksperimen dilakukan pada sesi berbeda, anggaran pencarian berbeda, dan GA memilih individu terbaik dari populasi awal, sedangkan HC memulai dari satu state; durasi HC juga tidak mencakup pembuatan state awal. Karena 15/18 run GA sudah optimum pada awal, keunggulan pencarian GA terhadap HC belum dapat digeneralisasi. Simulated Annealing menempati posisi kompromi yang seimbang antara kecepatan komputasi Hill-Climbing dan daya jelajah Genetic Algorithm. Dari sisi durasi, SA (0,0672 detik) sekitar 8 kali lebih cepat dibanding HC Sideways Move dan 15 kali lebih cepat dibanding GA.

Berbeda dengan Steepest Ascent dan Stochastic yang langsung terhenti pada plateau lokal awal, toleransi langkah buruk pada SA memungkinkan sistem keluar dari jebakan lokal sehingga menghasilkan kualitas solusi rata-rata yang lebih tinggi (1816,67). Namun demikian, pencapaian puncak absolut 1895 pada SA bergantung pada skema pendinginan; pendinginan yang terlalu cepat tanpa pencatatan solusi terbaik dapat menghentikan pencarian pada state di sekitar optimum.

Bab 3: Kesimpulan dan Saran

3.1 Kesimpulan

Representasi Package–Placement–State memisahkan atribut barang dari penempatannya, sedangkan validator menyatukan batas kontainer, tabrakan, dukungan, fragile, dan kapasitas. Objektif utama memaksimalkan nilai muatan; objektif alternatif menambahkan volume, urgensi ETA, dan preferensi peletakan rendah dengan batas proksi stabilitas yang telah dijelaskan.

Pada sampel 12 paket, Sideways mencapai optimum global 1.895 dalam tiga run. Random Restart mencapai nilai tersebut sekali dan memiliki rata-rata lebih tinggi daripada Steepest serta Stochastic, dengan tambahan biaya pencarian. Hasil ini menunjukkan manfaat eksplorasi plateau dan variasi state awal, tetapi belum cukup untuk generalisasi ke kasus lebih sulit.

GA mencapai optimum 1.895 pada 17/18 run dan seluruh state akhir valid; konfigurasi G=100 mencapai optimum pada seluruh pengulangan untuk N=20, 50, dan 100. Namun, 15 run sudah optimum sejak populasi awal, sehingga hasil terutama menunjukkan efektivitas inisialisasi pada sampel ini. Sideways menyamai kualitas dan konsistensi GA pembanding dengan durasi lebih rendah; memperbesar G atau N menambah waktu tanpa selalu memperbaiki solusi.

Skema pendinginan geometrik (alpha = 0,95) dan kriteria Metropolis terbukti efektif menyeimbangkan tahap eksplorasi global pada suhu tinggi dan eksploitasi lokal pada suhu rendah. Seluruh state akhir yang dihasilkan SA terbukti 100% valid terhadap batasan fisik 3D dan kapasitas peti kemas. Dengan durasi eksekusi rata-rata 0,0672 detik, SA mampu meloloskan diri dari jebakan lokal awal yang menghentikan varian Steepest Ascent dan Stochastic, menghasilkan nilai objektif akhir rata-rata 1816,67.

3.2 Saran

Perbaiki konsistensi log solusi terbaik dan penamaan berkas, lalu uji dengan seed berpasangan serta kasus yang mendekati batas volume/berat. Evaluasi batas sideways, jumlah restart, dan ukuran sampel tetangga secara terpisah; lengkapi visualisasi initial/final state serta plot konvergensi. Untuk kesesuaian operator, petakan rotasi 90° per sumbu secara eksplisit dan validasi skema masukan sebelum pencarian.

Untuk GA, teruskan rng sampai generator state dan gunakan seed berpasangan agar eksperimen dapat diulang secara utuh. Uji kasus lebih padat, bandingkan HC–GA dengan anggaran evaluasi setara, serta pertimbangkan perbaikan keturunan tidak valid dan penghentian ketika batas atas 1.895 tercapai. Catat populasi setelah reproduksi terakhir agar riwayat mencakup solusi keluaran, lalu lengkapi grafik dan visualisasi state.

Implementasikan mekanisme penyimpanan memori solusi terbaik (best-so-far tracking) agar konfigurasi yang dikembalikan sebagai hasil akhir adalah state dengan nilai tertinggi selama

pencarian, bukan sekadar posisi state pada langkah terakhir pendinginan. Selain itu, terapkan strategi pemanasan ulang (reheating) adaptif jika terdeteksi frekuensi penolakan berturut-turut pada suhu rendah, serta uji variasi laju pendinginan yang lebih landai (seperti alpha = 0,98 atau 0,99) dengan batas iterasi lebih besar untuk meningkatkan peluang konvergensi ke optimum global 1895.

Bab 4: Pembagian Tugas Anggota Kelompok

Nama / NIM	Implementasi	Eksperimen dan laporan
Timothy Bernard Soeharto 13524092	Model core, validator kendala 3D, operator perpindahan, parser/log JSON; empat varian Hill-Climbing.	Tiga run tiap varian HC; representasi, kendala, pembahasan core/HC dan analisisnya. Dokumentasi Bonus 1 yang sudah tersedia pada objective.py.
Muhammad Fatih Irkham Mauludi 13524004	Simulated Annealing; visualisasi 3D truk/paket dan grafik SA; rancangan awal objektif alternatif (opsional).	Simulated Annealing, Tiga run SA, plot probabilitas penerimaan dan analisis stuck; pembahasan SA serta visualisasi, pembuatan keseluruhan web app, deployment
Daniel Anindito Nugroho 13524002	Genetic Algorithm, benchmark CLI terpusat, dan automasi log eksperimen.	Matriks 18 run GA, grafik maksimum/rata-rata; analisis parameter dan perbandingan HC–SA–GA; kompilasi laporan, form AI, README.

Referensi

[1] M. L. Khodra, “Beyond Classical Search: Classical vs Local Search,” YouTube, 2021. [Video]. Tersedia: https://www.youtube.com/watch?v=oGTa9kGE3P8. [Diakses: 19 Oktober 2026].

[2] M. L. Khodra, “Beyond Classical Search: State (Value, Neighbor),” YouTube, 2021. [Video]. Tersedia: https://www.youtube.com/watch?v=_6rla37QorM. [Diakses: 19 Oktober 2026].

[3] M. L. Khodra, “Beyond Classical Search: Hill-climbing Search,” YouTube, 2021. [Video]. Tersedia: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Diakses: 19 Oktober 2026].

[4] M. L. Khodra, “Beyond Classical Search: Simulated Annealing,” YouTube, 2021. [Video]. Tersedia: https://www.youtube.com/watch?v=AdhG7HnhYrk. [Diakses: 19 Oktober 2026].

[5] M. L. Khodra, “Beyond Classical Search: Genetic Algorithm,” YouTube, 2021. [Video]. Tersedia: https://www.youtube.com/watch?v=aI41TGo9J9c. [Diakses: 19 Oktober 2026].

[6] Plotly Technologies Inc., “3D Mesh Plots in Python,” Plotly Python Documentation. [Online]. Tersedia: https://plotly.com/python/3d-mesh/. [Diakses: 19 Oktober 2026].

[7] Plotly Technologies Inc., “Dash Python User Guide,” Dash Documentation. [Online]. Tersedia: https://dash.plotly.com/. [Diakses: 19 Oktober 2026].

Lampiran: Form Penggunaan Generative AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM :
1. Daniel Anindito Nugroho / 13524002
2. Muhammad Fatih Irkham Mauludi / 13524004
3. Timothy Bernard Soeharto / 13524092
Fakultas/Sekolah : STEI-K ITB
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Optimasi Pemuatan Truk Peti Kemas 3D (Mejiro Express) Menggunakan Algoritma Local Search

Menyatakan bahwa kami menggunakan/ AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah: ChatGPT.

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Terbatas (1)
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Terbatas (2)
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Terbatas (3)
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya	Terbatas (4)
Lainnya, Jelaskan: bantuan sintaks/debugging Python serta pemeriksaan hasil eksperimen.	Ya	Terbatas (5)

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Daniel Anindito Nugroho
NIM 13524002



Muhammad Fatih Irkham Mauludi
NIM 13524004



Timothy Bernard Soeharto
NIM 13524092

Keterangan tingkat penggunaan

(1) Membantu proofreading, pemeriksaan ejaan/tata bahasa Bahasa Indonesia, dan perapian kalimat agar ringkas serta jelas.

(2) Membantu menyusun draf penjelasan dari kode dan merangkum hasil eksperimen nyata pada laporan ini.

(3) Membantu memperjelas konsep local search dan logika pengecekan geometri 3D serta membahas hasil eksperimen.

(4) Membuat visualisasi untuk *frontend* aplikasi Web App menggunakan *library* Dash dan Plotly pada Python

(5) Membantu referensi sintaks pustaka standar Python dan penelusuran bug/error bertahap ketika mengalami kendala teknis.